AI/ ML Engineer Assessment

ASSESSMENT INSTRUCTIONS

Position: AI/ ML Engineer

- **Start Time: 7th April 2026, 7:00 PM (IST)**
- **Deadline: 10th April 2026, 7:00 PM (IST)**
- **DEADLINE - NO EXTENSIONS GRANTED**

Submission Requirements:

- **File Name:** AI/ ML Engineer_[Your_Name] _April2026
- **Email to:** [career@purplemerit.com] with subject " AI/ML Engineer Assessment Submission - [Your Name]"

❖ **LATE SUBMISSIONS ARE DISQUALIFIED.**

Candidate Instructions (Assessment Selection & Tools)

- **We have provided two assessment options.** You may choose **either one** to complete and submit.
- **Completing both assessments is optional**; however, submitting **both** will be considered an **additional advantage**.
- You may use **any programming language** and **any frameworks/tools** you are comfortable with to implement your solution.
- You are encouraged to **add additional agents** beyond the minimum required to improve the outcome, robustness, or clarity of your system. *(Extra agents demonstrating thoughtful design will be positively evaluated.)*

ASSESSMENT 1

Build a multi-agent system that simulates a cross-functional "war room" during a product launch. The system must analyze a mock dashboard (metrics + user feedback) and produce a structured launch decision: **Proceed / Pause / Roll Back**, along with a concise action plan.

Scenario

PurpleMerit is rolling out a new feature. During the launch window, multiple metrics begin shifting and user feedback is mixed. A cross-functional war room is convened to decide whether to continue rollout, pause, or roll back and what actions must happen next.

Your system should simulate coordinated decision-making among these agents:

- **Product Manager (PM) Agent** — defines success criteria, user impact, and go/no-go framing
- **Data Analyst Agent** — analyses quantitative metrics, trends, anomalies, and confidence
- **Marketing/Comms Agent** — assesses messaging, customer perception, and communication actions
- **Risk/Critic Agent** — challenges assumptions, highlights risks, and requests additional evidence

You may **add additional agents if it improves outcomes.**

Inputs (Mock Dashboard)

Create a small but realistic mock dataset as inputs for your system. At minimum, include:

A) Metrics (time series for 7–14 days)
Include at least 6 of the following (or equivalents):

- Activation / Signup conversion
- DAU/WAU
- Retention (D1/D7)
- Crash rate / error rate
- API latency (e.g., p95)
- Payment success/failure rate
- Support ticket volume
- Feature adoption funnel completion
- Churn / cancellations

B) User Feedback (20–50 short entries)
A mix of positive/neutral/negative feedback, including repeated issues and a few outliers.

C) Release Notes / Known Issues
A short file describing the feature change and any known risks.

Required Capabilities (Must Have)

Your submission must include:

1. Multi-agent orchestration

- Clear separation of agent responsibilities
- A coordinator/orchestrator that manages the workflow and final decision
- Documented flow of how agents interact (in code and/or in the run trace)

2.Tool usage

- At least **two tools/functions** that agents invoke (e.g., metric aggregation, anomaly detection, sentiment summary, trend comparison)
- Tools must be called programmatically by agents

3. Structured final output
Produce a final **JSON or YAML** output containing at minimum:

- **Decision:** Proceed / Pause / Roll Back
- **Rationale:** key drivers (with metric references + feedback summary)
- **Risk register:** top risks with mitigation
- **Action plan (24–48 hours):** actions + owners
- **Communication plan:** internal/external messaging guidance
- **Confidence score** (and what would increase confidence)

4. Traceability

- Provide logs/traces showing agent steps and tool calls (console logs are acceptable)

Implementation Guidelines

- Preferred language: **Python** (recommended), but other languages are acceptable with clear instructions
- Any orchestration approach is acceptable (framework-based or custom)
- If an LLM key is required, it must be loaded via environment variables (no hard-coding)
- Focus on correctness, clarity, and engineering quality (UI is optional)

Submission Requirements (Mandatory)

Submit the following **two items**:

1. **GitHub Repository Link**

- Repository must contain the complete working solution
- Include a **README.md** with:
  - Setup instructions
  - How to run the program end-to-end
  - Example command(s) to reproduce the output
  - Any environment variables required

2. **Short Demo Video (No explanation required)**

- A short screen recording that **showcases the working output** of the assessment
- The video should show:
  - Running the system
  - The final structured output (JSON/YAML) produced by the system
- Do not include a verbal walkthrough or detailed explanation only a functional demo of the output

ASSESSMENT - 2

Build a multi-agent system that ingests a bug report and related logs, reproduces the issue by generating a **minimal reproducible test/script**, and outputs a **root-cause hypothesis** plus a **patch plan** (and optional patch).

Scenario

Engineering teams frequently receive bug reports with incomplete context. The goal of this assessment is to simulate an automated assistant that can:

1. understand the report, 2) extract evidence from logs, 3) attempt reproduction with code, and 4) propose a credible fix plan with verification steps.

What You Will Build

A runnable application (CLI recommended) that orchestrates multiple agents to perform the following workflow:

1. Parse inputs (bug report + logs + optional repository snapshot).
2. Triage and identify the most likely failure surface.
3. Generate and run a minimal reproduction script/test.
4. Summarize observed results and align them with evidence in logs.
5. Propose root cause + patch plan and how to validate the fix.

Agent Roles (Minimum Required)

Your system must include at least the following agents:

- **Triage Agent**
    - Extracts key symptoms, expected vs actual behavior, environment details, and prioritizes hypotheses.
- **Log Analyst Agent**
    - Searches logs for stack traces, error signatures, frequency, correlation with deploy/version, and key anomalies.
- **Reproduction Agent**
    - Constructs a minimal reproduction script/test (unit test/integration test) and runs it using tools.
- **Fix Planner Agent**
    - Proposes a root-cause hypothesis, patch approach, and verification plan. Must reference reproduction outcome and log evidence.
- **Reviewer/Critic Agent** *(required)*

- Challenges weak assumptions, checks whether the repro is truly minimal, verifies the fix plan is safe, and suggests edge cases.

You may add additional agents (e.g., “Repo Navigator”, “Dependency Analyst”) if helpful.

Inputs

You may choose **one** of the following input modes (declare clearly in your README):

Option A (Recommended): Provided Mini-Repo

Use a small codebase (your own or a prepared sample) with an intentionally introduced bug. Include:

- Bug report (Markdown or JSON)
- Logs (plain text)
- Steps to run the app/tests

Option B: Log + Report Only (No Repo)

If you do not include a codebase, your system must still:

- Generate a minimal reproduction in a standalone script that simulates the failure
- Provide a patch plan at the design level (no code patch required)

Bug report must include:

- Title, description
- Expected vs actual behavior
- Environment (language/runtime version, OS if relevant)
- Reproduction hints (even if incomplete)

Logs must include:

- At least one stack trace or clear error signature
- A few red herrings/noise lines to test robustness

Required Capabilities (Must Have)

1. Multi-agent orchestration

- Clear separation of responsibilities and a coordinator/orchestrator
- Deterministic workflow (graph/state machine encouraged but not mandatory)
- Explicit handoffs between agents

2. Tool usage (execution required)
Agents must programmatically call tools such as:

- ripgrep/file search over logs/repo

- Running tests or scripts (pytest, node, java, etc.)
- Parsing logs / extracting stack traces
- (Optional) static analysis or dependency inspection

3. Minimal reproducible artifact

- Output a runnable repro: a small script/test file that fails consistently
- Include how to run it (command, expected failing output)

4. Structured outputs
Generate a final **JSON or YAML** file containing at minimum:

- Bug summary (symptoms, scope, severity)
- Evidence (log lines/stack trace excerpts)
- Repro steps + repro artifact path
- Root-cause hypothesis (with confidence)
- Patch plan (files/modules impacted, approach, risks)
- Validation plan (tests to add, regression checks)
- Open questions / missing info

5. Traceability

- Provide logs/traces of agent decisions and tool calls (console logs acceptable)
- Include a short note in README on where traces are located and how to read them

Implementation Guidelines

- Preferred language: **Python** (recommended), but any language is acceptable with clear instructions
- Any agent framework is acceptable (LangGraph / AutoGen / CrewAI / custom)
- Do not hard-code secrets; use environment variables if required
- Prioritize correctness, reproducibility, and engineering quality over UI

Deliverables

Submit:

1. **GitHub repository link** containing:
    - Source code
    - Input artifacts (bug report, logs, and optionally the mini-repo)
    - Repro test/script generated by the system (or generated on first run and saved)

- **README.md** with setup + run instructions

2. **Short demo video (no explanation required)**
    - Screen recording showing:
        - Running the system end-to-end
        - The minimal repro failing
        - The final structured output file being generated (JSON/YAML)

Good luck—we're looking forward to seeing how you approach this AI Engineer challenge. There's no single "right" solution; we'll be evaluating your engineering judgment, sound system design, reproducibility, clear evaluation, attention to safety and failure modes, and your ability to deliver a reliable, high-quality outcome.